JAVA

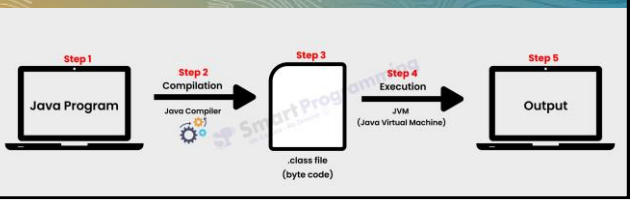
DESENVOLVIMENTO DE APLICAÇÕES

Prof. Leonardo Coelho | 2026

TEMA 02







1

1. Método

Pense num método como uma máquina ou uma mini receita dentro do seu programa.

- Ela tem um nome.
- Ela pode receber ingredientes (chamados de parâmetros).
- Ela executa uma tarefa isolada.
- Ela pode te devolver um resultado (chamado de retorno).

Exemplo Prático: Criando seu próprio método

Até agora, você usou o método main (que o Java cria) e o método System.out.println (que já vem pronto no Java).

Agora você vai criar o seu.

Imagine um método que recebe dois números, soma eles e devolve o resultado.

Veja como escrever isso fora do main, mas ainda dentro da classe Main:

2

DESENVOLVIMENTO DE APLICAÇÕES

1. Método

```java
public class Main {

    // 1. Este é o método principal (o ponto de partida)
    public static void main(String[] args) {

        // Chamando o nosso método personalizado e guardando a resposta
        int resultadoDaSoma = somarDoisNumeros(10, 5);

        System.out.println("O resultado da soma é: " + resultadoDaSoma);
    }

    // 2. Este é o SEU método personalizado (a maquininha)
    public static int somarDoisNumeros(int numero1, int numero2) {
        int total = numero1 + numero2;
        return total; // "return" envia o resultado de volta para quem chamou
    }
}
```

3

DESENVOLVIMENTO DE APLICAÇÕES

1. Método

Entendendo a estrutura do seu método:

public static int somarDoisNumeros(...)

public static: Por enquanto, usamos isso para que o método main consiga enxergar e usar esse novo método.

int: É o tipo de retorno. Significa: "Esta máquina promete devolver um número inteiro quando terminar de rodar". (Se não fosse devolver nada, usaríamos void).

somarDoisNumeros: É o nome que você escolheu para o seu método.

(int numero1, int numero2): São as variáveis de entrada (os ingredientes que a máquina precisa para funcionar).

4

DESENVOLVIMENTO DE APLICAÇÕES

1. Método (REUTILIZAÇÃO)

A maior vantagem é que você pode usar essa "máquina" quantas vezes quiser com valores diferentes, sem precisar reescrever a lógica da soma:

```java
int compra1 = somarDoisNumeros(50, 20); // Devolve 70
int compra2 = somarDoisNumeros(100, 5); // Devolve 105
```

5

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR

Atividade 1: O Conversor de Moedas (Método com Retorno)

O Cenário: Você está desenvolvendo um mini sistema para uma agência de viagens e precisa de uma função que converta valores em Reais (BRL) para Dólares (USD).

O que o seu código deve fazer:

- Fora do método main, crie um método chamado converterRealParaDolar.
- Esse método deve **receber** dois parâmetros: double valorEmReal e double cotacaoDoDolar.
- O método deve **calcular** o valor convertido (Valor em Real dividido pela Cotação) e **retornar** (return) esse resultado do tipo double.
- Dentro do seu método main, você deve chamar o método que criou passando um valor em reais (ex: 1500.00) e a cotação do dia (ex: 5.25), armazenar o resultado numa variável e exibir na tela com o System.out.println.



6

DESENVOLVIMENTO DE APLICAÇÕES

```
public class Main {

    // 1. O MÉTODO PRINCIPAL (Ponto de partida do programa)
    public static void main(String[] args) {
        double meuDinheiroEmReal = 1500.00;
        double cotacaoDoDia = 5.25;

        // Chamamos o método passando os valores e guardamos o resultado na variável 'resultado'
        double resultado = converterRealParaDolar(meuDinheiroEmReal, cotacaoDoDia);

        // Exibe o resultado na tela
        System.out.println("Com R$ " + meuDinheiroEmReal + " você consegue comprar U$ " + resultado);
    }

    // 2. O SEU MÉTODO PERSONALIZADO (A maquininha de conversão)
    // Ele promete retornar um 'double' e precisa receber dois valores 'double' para funcionar
    public static double converterRealParaDolar(double valorEmReal, double cotacaoDoDolar) {
        double valorConvertido = valorEmReal / cotacaoDoDolar;

        return valorConvertido; // Envia o resultado de volta para o 'main'
    }
}
```

7

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR

Atividade 2: Verificador de Login (Método que Executa Ação / Boolean)

O Cenário: Você está criando a tela de login de um aplicativo e precisa validar se o usuário e a senha digitados estão corretos.

O que o seu código deve fazer:

1. Fora do método main, crie um método chamado validarLogin.
2. Esse método deve **receber** duas String como parâmetro: String usuario e String senha.
3. O método deve retornar um valor **boolean** (true se o login for válido ou false se for inválido).
4. **Regra de validação dentro do método:** Para comparar textos em Java, não usamos ==. Usamos o comando .equals().
    1. Exemplo: usuario.equals("admin")
5. Defina que o usuário correto é "admin" e a senha correta é "12345". Use o if dentro do método para verificar se **ambos** estão corretos. Se sim, retorne true, senão retorne false.
6. No seu método main, chame esse método passando um usuário e senha quaisquer. Use um if no main para exibir "Acesso Permitido!" se o método retornar true, ou "Usuário ou senha incorretos!" se retornar false.



8

DESENVOLVIMENTO DE APLICAÇÕES

Atividade 2: Verificador de Login (Método que Executa Ação / Boolean)

```
public class Main {
  // 1. O MÉTODO PRINCIPAL
  public static void main(String[] args) {
    // Tentativa de login (Experimente mudar a senha para "errada" para ver o outro resultado!)
    String usuarioDigitado = "admin";
    String senhaDigitada = "12345";

    // Chamamos o método que criamos. Ele vai nos devolver 'true' ou 'false'
    boolean loginAutorizado = validarLogin(usuarioDigitado, senhaDigitada);

    // Usamos um IF para tomar a decisão baseada no retorno do método
    if (loginAutorizado == true) {
      System.out.println("Acesso Permitido! Bem-vindo ao sistema.");
    } else {
      System.out.println("Acesso Negado! Usuário ou senha incorretos.");
    }
  }
```

9

DESENVOLVIMENTO DE APLICAÇÕES

Atividade 2: Verificador de Login (Método que Executa Ação / Boolean)

```
  // 2. O SEU MÉTODO PERSONALIZADO (A maquininha de validação)
  // Ele promete retornar um 'boolean' (true/false) e recebe dois textos (Strings)
  public static boolean validarLogin(String usuario, String senha) {

    // Verificamos se o usuário É "admin" E a senha É "12345"
    if (usuario.equals("admin") && senha.equals("12345")) {
      return true;  // Se ambos estiverem certos, retorna verdadeiro
    } else {
      return false; // Se algo estiver errado, retorna falso
    }
  }
}
```

10

DESENVOLVIMENTO DE APLICAÇÕES

DICA IMPORTANTE – CASAS DECIMAIS (DOUBLE)

Para diminuir ou formatar as casas decimais de uma variável do tipo double em Java (por exemplo, transformar 2.5333333 em apenas 2.53), a forma mais simples e utilizada é o método **System.out.printf**.

A letra **f** no final significa *Formatted* (Formatado). Esse comando permite-nos criar uma regra para o texto antes de o exibir no ecrã.

```java
public class Main {

    public static void main(String[] args) {
        double notaFinal = 7.666666666666667;

        System.out.println("Sem formatação: " + notaFinal);

        System.out.printf("Com duas casas: %.2f\n", notaFinal);

        System.out.printf("Com uma casa: %.1f\n", notaFinal);
    }
}
```

```
Sem formatação: 7.666666666666667
Com duas casas: 7.67
Com uma casa: 7.7
```

11

DESENVOLVIMENTO DE APLICAÇÕES

DICA IMPORTANTE – CASAS DECIMAIS (DOUBLE)

- `%` : Avisa o Java: *"Atenção, aqui vai entrar uma variável!"*.
- `.2` : Define o número de casas decimais que queres mostrar (podes mudar para `.1`, `.3`, `.4`, etc.).
- `f` : Diz ao Java que a variável que vai ali é um número de ponto flutuante (`double` ou `float`).
- `\n` : É um comando para **pular uma linha**. Como o `printf` não pula de linha automaticamente (ao contrário do `println`), precisamos de colocar o `\n` no final do texto para que a próxima mensagem não fique colada na mesma linha.

💡 **Nota de Arredondamento:** O Java é inteligente! No exemplo acima, ao pedir duas casas decimais para `7.6666...` , ele vai arredondar automaticamente para `7.67` .

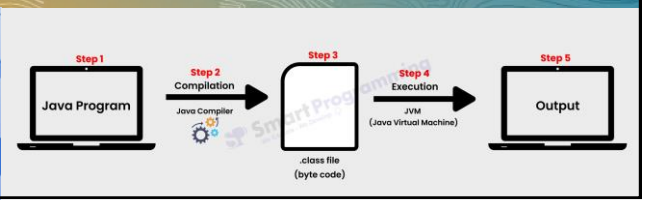
12

JAVA

DESENVOLVIMENTO DE APLICAÇÕES







13